1. class Graph {

```
// inner class
// to keep track of edges
class Edge {
  int src, dest;
}

// number of vertices and edges
int vertices, edges;

// array to store all edges
Edge[] edge;

Graph(int vertices, int edges) {
  this.vertices = vertices;
  this.edges = edges;

  // initialize the edge array
  edge = new Edge[edges];
  for(int i = 0; i < edges; i++) {

    // each element of the edge array
    // is an object of Edge type
    edge[i] = new Edge();
  }
}

public static void main(String[] args) {

  // create an object of Graph class
  int noVertices = 5;
  int noEdges = 8;
  Graph g = new Graph(noVertices, noEdges);

  // create graph
  g.edge[0].src = 1;   // edge 1---2
  g.edge[0].dest = 2;

  g.edge[1].src = 1;   // edge 1---3
  g.edge[1].dest = 3;

  g.edge[2].src = 1;   // edge 1---4
  g.edge[2].dest = 4;
```

```
    g.edge[3].src = 2;   // edge 2---4
    g.edge[3].dest = 4;

    g.edge[4].src = 2;   // edge 2---5
    g.edge[4].dest = 5;

    g.edge[5].src = 3;   // edge 3---4
    g.edge[5].dest = 4;

    g.edge[6].src = 3;   // edge 3---5
    g.edge[6].dest = 5;

    g.edge[7].src = 4;   // edge 4---5
    g.edge[7].dest = 5;

    // print graph
    for(int i = 0; i < noEdges; i++) {
      System.out.println(g.edge[i].src + " - " + g.edge[i].dest);
    }

  }
}
```

```
1 - 2
1 - 3
1 - 4
2 - 4
2 - 5
3 - 4
3 - 5
4 - 5

=== Code Execution Successful ===
```

2. import java.util.ArrayList;

```
import java.util.List;

public class JoinLists {

    public static void main(String[] args) {

        List<String> list1 = new ArrayList<String>();
        list1.add("a");
```

```
        List<String> list2 = new ArrayList<String>();
        list2.add("b");

        List<String> joined = new ArrayList<String>();

        joined.addAll(list1);
        joined.addAll(list2);

        System.out.println("list1: " + list1);
        System.out.println("list2: " + list2);
        System.out.println("joined: " + joined);

    }
}
```

```
list1: [a]
list2: [b]
joined: [a, b]

=== Code Execution Successful ===
```

3. import java.util.ArrayList;

```
class Main {
  public static void main(String[] args) {
    ArrayList languages = new ArrayList<>();

    // Add elements in the list
    languages.add("Java");
    languages.add("Python");
    languages.add("JavaScript");
    System.out.println("ArrayList: " + languages);

    // Create a new array of String type
    String[] arr = new String[languages.size()];

    // Convert ArrayList into the string array
    languages.toArray(arr);
    System.out.print("Array: ");
    for(String item:arr) {
```

```
        System.out.print(item+", ");
    }
  }
}
```

```
Note: Main.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
ArrayList: [Java, Python, JavaScript]
Array: Java, Python, JavaScript,
=== Code Execution Successful ===
```

4.
```
import java.util.Arrays;
import java.util.ArrayList;
import java.util.List;

class Main {
  public static void main(String[] args) {

    // create an array
    String[] array = {"Java", "Python", "C"};
    System.out.println("Array: " + Arrays.toString(array));

    // convert array to list
    List languages= new ArrayList<>(Arrays.asList(array));

    System.out.println("List: " + languages);

  }
}
```

```
Array: [Java, Python, C]
List: [Java, Python, C]

=== Code Execution Successful ===
```

5.
```
public class Prime {
```

```
    public static void main(String[] args) {

        int low = 20, high = 50;

        while (low < high) {
            boolean flag = false;

            for(int i = 2; i <= low/2; ++i) {
                // condition for nonprime number
                if(low % i == 0) {
                    flag = true;
                    break;
                }
            }

            if (!flag && low != 0 && low != 1)
                System.out.print(low + " ");

            ++low;
        }
    }
}
```

```
23 29 31 37 41 43 47
=== Code Execution Successful ===
```

6. public class Factorial {

```
    public static void main(String[] args) {
        int num = 6;
        long factorial = multiplyNumbers(num);
        System.out.println("Factorial of " + num + " = " + factorial);
    }
    public static long multiplyNumbers(int num)
    {
        if (num >= 1)
            return num * multiplyNumbers(num - 1);
        else
```

```
        return 1;
    }
}
```

```
Factorial of 6 = 720


=== Code Execution Successful ===
```

```
7. class Main {

  public static void main(String[] args) {

    // binary number
    long num = 110110111;

    // call method by passing the binary number
    int decimal = convertBinaryToDecimal(num);

    System.out.println("Binary to Decimal");
    System.out.println(num + " = " + decimal);
  }

  public static int convertBinaryToDecimal(long num) {
    int decimalNumber = 0, i = 0;
    long remainder;

    while (num != 0) {
      remainder = num % 10;
      num /= 10;
      decimalNumber += remainder * Math.pow(2, i);
      ++i;
    }

    return decimalNumber;
  }
}
```

```
Binary to Decimal
110110111 = 439

=== Code Execution Successful ===
```

```
8. class Main {
  public static void main(String[] args) {

    // binary number
    String binary = "01011011";

    // convert to decimal
    int decimal = Integer.parseInt(binary, 2);
    System.out.println(binary + " in binary = " + decimal + " in decimal.");
  }
}
```

```
01011011 in binary = 91 in decimal.

=== Code Execution Successful ===
```

```
9. public class DecimalOctal {

    public static void main(String[] args) {
        int decimal = 78;
        int octal = convertDecimalToOctal(decimal);
        System.out.printf("%d in decimal = %d in octal", decimal, octal);
    }

    public static int convertDecimalToOctal(int decimal)
    {
        int octalNumber = 0, i = 1;

        while (decimal != 0)
        {
            octalNumber += (decimal % 8) * i;
            decimal /= 8;
```

```
            i *= 10;
        }

        return octalNumber;
    }
}
```

```
78 in decimal = 116 in octal
=== Code Execution Successful ===
```

10. public class OctalDecimal {

```
    public static void main(String[] args) {
        int octal = 116;
        int decimal = convertOctalToDecimal(octal);
        System.out.printf("%d in octal = %d in decimal", octal, decimal);
    }

    public static int convertOctalToDecimal(int octal)
    {
        int decimalNumber = 0, i = 0;

        while(octal != 0)
        {
            decimalNumber += (octal % 10) * Math.pow(8, i);
            ++i;
            octal/=10;
        }

        return decimalNumber;
    }
}
```

```
116 in octal = 78 in decimal
=== Code Execution Successful ===
```

11. public class Main {

```
  public static void main(String[] args) {
    int number = 34;
    boolean flag = false;
    for (int i = 2; i <= number / 2; ++i) {

      // condition for i to be a prime number
      if (checkPrime(i)) {

        // condition for n-i to be a prime number
        if (checkPrime(number - i)) {

          // n = primeNumber1 + primeNumber2
          System.out.printf("%d = %d + %d\n", number, i, number - i);
          flag = true;
        }

      }
    }

    if (!flag)
      System.out.println(number + " cannot be expressed as the sum of two prime numbers.");
  }

  // Function to check prime number
  static boolean checkPrime(int num) {
    boolean isPrime = true;

    for (int i = 2; i <= num / 2; ++i) {
      if (num % i == 0) {
        isPrime = false;
        break;
      }
    }
```

```
    return isPrime;
  }
}
```

```
34 = 3 + 31
34 = 5 + 29
34 = 11 + 23
34 = 17 + 17

=== Code Execution Successful ===
```

12. public class Armstrong {

```
    public static void main(String[] args) {

        int low = 999, high = 99999;

        for(int number = low + 1; number < high; ++number) {

            if (checkArmstrong(number))
                System.out.print(number + " ");
        }
    }

    public static boolean checkArmstrong(int num) {
        int digits = 0;
        int result = 0;
        int originalNumber = num;

        // number of digits calculation
        while (originalNumber != 0) {
            originalNumber /= 10;
            ++digits;
        }

        originalNumber = num;

        // result contains sum of nth power of its digits
        while (originalNumber != 0) {
            int remainder = originalNumber % 10;
            result += Math.pow(remainder, digits);
            originalNumber /= 10;
        }
```

```
        if (result == num)
            return true;

        return false;
    }
}
```

```
1634 8208 9474 54748 92727 93084
=== Code Execution Successful ===
```

13. public class Prime {

```
    public static void main(String[] args) {

        int low = 20, high = 50;

        while (low < high) {
            if(checkPrimeNumber(low))
                System.out.print(low + " ");

            ++low;
        }
    }

    public static boolean checkPrimeNumber(int num) {
        boolean flag = true;

        for(int i = 2; i <= num/2; ++i) {

            if(num % i == 0) {
                flag = false;
                break;
            }
        }

        return flag;
    }
}
```

```
23 29 31 37 41 43 47
=== Code Execution Successful ===
```

14 . import java.util.Scanner;

```
class Main {
  public static void main(String[] args) {

    char operator;
    Double number1, number2, result;

    // create an object of Scanner class
    Scanner input = new Scanner(System.in);

    // ask users to enter operator
    System.out.println("Choose an operator: +, -, *, or /");
    operator = input.next().charAt(0);

    // ask users to enter numbers
    System.out.println("Enter first number");
    number1 = input.nextDouble();

    System.out.println("Enter second number");
    number2 = input.nextDouble();

    switch (operator) {

      // performs addition between numbers
      case '+':
        result = number1 + number2;
        System.out.println(number1 + " + " + number2 + " = " + result);
        break;

      // performs subtraction between numbers
      case '-':
        result = number1 - number2;
        System.out.println(number1 + " - " + number2 + " = " + result);
        break;
```

```
      // performs multiplication between numbers
      case '*':
        result = number1 * number2;
        System.out.println(number1 + " * " + number2 + " = " + result);
        break;

      // performs division between numbers
      case '/':
        result = number1 / number2;
        System.out.println(number1 + " / " + number2 + " = " + result);
        break;

      default:
        System.out.println("Invalid operator!");
        break;
    }

    input.close();
  }
}
```

```
Choose an operator: +, -, *, or /
+
Enter first number
4
Enter second number
90
4.0 + 90.0 = 94.0

=== Code Execution Successful ===
```

15. public class StandardDeviation {

```
  public static void main(String[] args) {
    double[] numArray = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    double SD = calculateSD(numArray);

    System.out.format("Standard Deviation = %.6f", SD);
  }

  public static double calculateSD(double numArray[])
```

```
    {
        double sum = 0.0, standardDeviation = 0.0;
        int length = numArray.length;

        for(double num : numArray) {
            sum += num;
        }

        double mean = sum/length;

        for(double num: numArray) {
            standardDeviation += Math.pow(num - mean, 2);
        }

        return Math.sqrt(standardDeviation/length);
    }
}
```

```
Standard Deviation = 2.872281
=== Code Execution Successful ===
```

16.
```
public class MultiplyMatrices {

    public static void main(String[] args) {
        int r1 = 2, c1 = 3;
        int r2 = 3, c2 = 2;
        int[][] firstMatrix = { {3, -2, 5}, {3, 0, 4} };
        int[][] secondMatrix = { {2, 3}, {-9, 0}, {0, 4} };

        // Mutliplying Two matrices
        int[][] product = new int[r1][c2];
        for(int i = 0; i < r1; i++) {
            for (int j = 0; j < c2; j++) {
                for (int k = 0; k < c1; k++) {
                    product[i][j] += firstMatrix[i][k] * secondMatrix[k][j];
                }
            }
        }
```

```
        // Displaying the result
        System.out.println("Multiplication of two matrices is: ");
        for(int[] row : product) {
            for (int column : row) {
                System.out.print(column + "    ");
            }
            System.out.println();
        }
    }
}
```

```
Multiplication of two matrices is:
24    29
6    25

=== Code Execution Successful ===
```

17. public class MultiplyMatrices {

```
    public static void main(String[] args) {
        int r1 = 2, c1 = 3;
        int r2 = 3, c2 = 2;
        int[][] firstMatrix = { {3, -2, 5}, {3, 0, 4} };
        int[][] secondMatrix = { {2, 3}, {-9, 0}, {0, 4} };

        // Mutliplying Two matrices
        int[][] product = multiplyMatrices(firstMatrix, secondMatrix, r1, c1, c2);

        // Displaying the result
        displayProduct(product);
    }

    public static int[][] multiplyMatrices(int[][] firstMatrix, int[][] secondMatrix, int r1, int c1, int c2) {
        int[][] product = new int[r1][c2];
        for(int i = 0; i < r1; i++) {
            for (int j = 0; j < c2; j++) {
                for (int k = 0; k < c1; k++) {
                    product[i][j] += firstMatrix[i][k] * secondMatrix[k][j];
                }
            }
```

```
    }

    return product;
  }

  public static void displayProduct(int[][] product) {
    System.out.println("Product of two matrices is: ");
    for(int[] row : product) {
      for (int column : row) {
        System.out.print(column + "    ");
      }
      System.out.println();
    }
  }
}
```

```
Product of two matrices is:
24    29
6    25

=== Code Execution Successful ===
```

18.
```
public class MultiplyMatrices {

  public static void main(String[] args) {
    int r1 = 2, c1 = 3;
    int r2 = 3, c2 = 2;
    int[][] firstMatrix = { {3, -2, 5}, {3, 0, 4} };
    int[][] secondMatrix = { {2, 3}, {-9, 0}, {0, 4} };

    // Mutliplying Two matrices
    int[][] product = multiplyMatrices(firstMatrix, secondMatrix, r1, c1, c2);

    // Displaying the result
    displayProduct(product);
  }

  public static int[][] multiplyMatrices(int[][] firstMatrix, int[][] secondMatrix, int r1, int c1, int c2) {
```

```
        int[][] product = new int[r1][c2];
        for(int i = 0; i < r1; i++) {
            for (int j = 0; j < c2; j++) {
                for (int k = 0; k < c1; k++) {
                    product[i][j] += firstMatrix[i][k] * secondMatrix[k][j];
                }
            }
        }

        return product;
    }

    public static void displayProduct(int[][] product) {
        System.out.println("Product of two matrices is: ");
        for(int[] row : product) {
            for (int column : row) {
                System.out.print(column + "    ");
            }
            System.out.println();
        }
    }
}
```

```
roduct of two matrices is:
4    29
    25

== Code Execution Successful ===
```

19. public class Array {

```
    public static void main(String[] args) {
        int[] array = {1, 2, 3, 4, 5};

        for (int element: array) {
            System.out.println(element);
        }
    }
}
```

```
1
2
3
4
5

=== Code Execution Succes
```

20. public class Main {

```
  public static void main(String[] args) {
    int number = 34;
    boolean flag = false;
    for (int i = 2; i <= number / 2; ++i) {

      // condition for i to be a prime number
      if (checkPrime(i)) {

        // condition for n-i to be a prime number
        if (checkPrime(number - i)) {

          // n = primeNumber1 + primeNumber2
          System.out.printf("%d = %d + %d\n", number, i, number - i);
          flag = true;
        }

      }
    }

    if (!flag)
      System.out.println(number + " cannot be expressed as the sum of two prime numbers.");
  }

  // Function to check prime number
  static boolean checkPrime(int num) {
    boolean isPrime = true;

    for (int i = 2; i <= num / 2; ++i) {
      if (num % i == 0) {
        isPrime = false;
        break;
      }
```

```
    }

    return isPrime;
  }
}
```

```
Output

34 = 3 + 31
34 = 5 + 29
34 = 11 + 23
34 = 17 + 17

=== Code Execution Successful ===
```

```
21. import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class TimeString {

    public static void main(String[] args) {
        // Format y-M-d or yyyy-MM-d
        String string = "2017-07-25";
        LocalDate date = LocalDate.parse(string, DateTimeFormatter.ISO_DATE);

        System.out.println(date);
    }
}
```

```
2017-07-25

=== Code Execution Successful ===
```

```
22. import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.Locale;
```

```
public class TimeString {

    public static void main(String[] args) {
        String string = "July 25, 2017";

        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("MMMM d, yyyy",
Locale.ENGLISH);
        LocalDate date = LocalDate.parse(string, formatter);

        System.out.println(date);
    }
}
```

```
2017-07-25

=== Code Execution Successful ==
```

```
 23.import java.util.Arrays;

class Main {
  public static void main(String[] args) {
    String str1 = "Race";
    String str2 = "Care";

    str1 = str1.toLowerCase();
    str2 = str2.toLowerCase();

    // check if length is same
    if(str1.length() == str2.length()) {

      // convert strings to char array
      char[] charArray1 = str1.toCharArray();
      char[] charArray2 = str2.toCharArray();

      // sort the char array
      Arrays.sort(charArray1);
      Arrays.sort(charArray2);

      // if sorted char arrays are same
```

```
      // then the string is anagram
      boolean result = Arrays.equals(charArray1, charArray2);

      if(result) {
        System.out.println(str1 + " and " + str2 + " are anagram.");
      }
      else {
        System.out.println(str1 + " and " + str2 + " are not anagram.");
      }
    }
    else {
      System.out.println(str1 + " and " + str2 + " are not anagram.");
    }
  }
}
```

```
race and care are anagram.

=== Code Execution Successful ===
```

24. import java.util.Arrays;

```
import java.util.Scanner;

class Main {
  public static void main(String[] args) {

    // create an object of Scanner class
    Scanner input = new Scanner(System.in);

    // take input from users
    System.out.print("Enter first String: ");
    String str1 = input.nextLine();
    System.out.print("Enter second String: ");
    String str2 = input.nextLine();

    // check if length is same
    if(str1.length() == str2.length()) {
```

```
      // convert strings to char array
      char[] charArray1 = str1.toCharArray();
      char[] charArray2 = str2.toCharArray();

      // sort the char array
      Arrays.sort(charArray1);
      Arrays.sort(charArray2);

      // if sorted char arrays are same
      // then the string is anagram
      boolean result = Arrays.equals(charArray1, charArray2);

      if(result) {
        System.out.println(str1 + " and " + str2 + " are anagram.");
      }
      else {
        System.out.println(str1 + " and " + str2 + " are not anagram.");
      }
    }
    else {
      System.out.println(str1 + " and " + str2 + " are not anagram.");
    }

    input.close();
  }
}
```

```
:nter first String: harini
:nter second String: justin
ıarini and justin are not anagram.

=== Code Execution Successful ===
```

25. import java.io.*;

```
public class OutputStreamString {

    public static void main(String[] args) throws IOException {

        ByteArrayOutputStream stream = new ByteArrayOutputStream();
        String line = "Hello there!";
```

```
        stream.write(line.getBytes());
        String finalString = new String(stream.toByteArray());

        System.out.println(finalString);
    }
```

```
Hello there!

=== Code Execution Successful ===
```

26. class Main {

```
  public static void main(String[] args) {

    // create string variables
    String str1 = "23";
    String str2 = "4566";

    // convert string to int
    // using parseInt()
    int num1 = Integer.parseInt(str1);
    int num2 = Integer.parseInt(str2);

    // print int values
    System.out.println(num1);    // 23
    System.out.println(num2);    // 4566
  }
}
```

```
23
4566

=== Code Execution Successful ===
```

27. public class CharString {

```
    public static void main(String[] args) {
        char ch = 'c';
        String st = Character.toString(ch);
```

```
        // Alternatively
        // st = String.valueOf(ch);

        System.out.println("The string is: " + st);
    }
}
```

```
The string is: c

=== Code Execution Successful ===
```

28. public class CharString {

```
    public static void main(String[] args) {
        char[] ch = {'a', 'e', 'i', 'o', 'u'};

        String st = String.valueOf(ch);
        String st2 = new String(ch);

        System.out.println(st);
        System.out.println(st2);
    }
}
```

```
aeiou
aeiou

=== Code Execution Successful ===
```

29. import java.util.Arrays;

```
public class StringChar {

    public static void main(String[] args) {
        String st = "This is great";
```

```
        char[] chars = st.toCharArray();
        System.out.println(Arrays.toString(chars));
    }
}
```

```
[T, h, i, s,  , i, s,  , g, r, e, a, t]

=== Code Execution Successful ===
```

30. // importing the File class

```
import java.io.File;

class Main {
  public static void main(String[] args) {

    // create a file object for the current location
    File file = new File("JavaFile.java");

    try {

      // create a new file with name specified
      // by the file object
      boolean value = file.createNewFile();
      if (value) {
        System.out.println("New Java File is created.");
      }
      else {
        System.out.println("The file already exists.");
      }
    }
    catch(Exception e) {
      e.getStackTrace();
    }
  }
}
```

```
New Java File is created.

=== Code Execution Successful ===
```

31.
```
// importing the FileWriter class
import java.io.FileWriter;

class Main {
  public static void main(String args[]) {

    // creates a multiline string using + operator
    // the string is a Java Program
    String program = "class JavaFile { " +
                    "public static void main(String[] args) { " +
                      "System.out.println(\"This is file\");"+
                    "}"+
                  "}";
     try {
       // Creates a Writer using FileWriter
       FileWriter output = new FileWriter("JavaFile.java");

       // Writes the program to file
       output.write(program);
       System.out.println("Data is written to the file.");

       // Closes the writer
       output.close();
     }
     catch (Exception e) {
       e.getStackTrace();
     }
  }
}
```

```
Data is written to the file.

=== Code Execution Successful ===
```

32. import java.io.File;

```
class Main {
  public static void main(String[] args) {

    try {
      // create a new file object
      File directory = new File("Directory");

      // delete the directory
      boolean result = directory.delete();

      if(result) {
        System.out.println("Directory Deleted");
      }
      else {
        System.out.println("Directory not Found");
      }

    } catch (Exception e) {
      e.getStackTrace();
    }
  }
}
```

```
Directory not Found

=== Code Execution Successful ===
```